**Université Hassan II de Casablanca**

Faculté des Sciences Aïn Chock (FSAC)

# Projet 5

## Calibration d'un Capteur de Qualité d'Air

Régression Linéaire Supervisée

Dataset : Air Quality UCI — Milan, Italie

**Réalisé par :** Khalifa HTE
Brahim SAOUAB
Badr EL MASSAOUDI

**Encadré par :** Pr. Khadija Tlemcani

FSAC, Université Hassan II de Casablanca

Année universitaire : 2025–2026

# 0 Table des matières

**1 Introduction** **2**
1.1 Contexte . . . 2
1.2 Objectifs . . . 2

**2 Dataset — Air Quality UCI** **2**
2.1 Description . . . 2
2.2 Valeurs manquantes . . . 2
2.3 Pourquoi CO(GT) comme référence ? . . . 2

**3 Prétraitement des données** **3**
3.1 Chargement . . . 3
3.2 Normalisation . . . 3

**4 Modèle de Machine Learning** **3**
4.1 Type de projet . . . 3
4.2 Split 80 % / 20 % . . . 4
4.3 Régression Linéaire . . . 4

**5 Évaluation du modèle** **4**
5.1 Métriques utilisées . . . 5
5.2 Indicateurs de fiabilité . . . 5

**6 Visualisations** **5**
6.1 Code complet des graphiques . . . 5
6.2 Analyse des graphiques . . . 7

**7 Bibliothèques Python utilisées** **7**
7.1 NumPy — Calcul numérique . . . 7
7.2 Pandas — Manipulation de données . . . 8
7.3 Scikit-learn — Machine Learning . . . 9
7.4 Matplotlib — Visualisation . . . 10
7.5 Récapitulatif des bibliothèques . . . 12

**8 Conclusion** **12**

# 1 Introduction

Ce projet s'inscrit dans le domaine du Machine Learning supervisé appliqué à la calibration de capteurs environnementaux. L'objectif est de corriger le biais systématique d'un capteur de qualité d'air bas coût (`PT08.S1`) en le comparant à un analyseur de référence certifié (`CO(GT)`).

## 1.1 Contexte

La surveillance de la qualité de l'air est un enjeu majeur de santé publique. Les capteurs bas coût permettent un déploiement massif mais souffrent d'imprécisions. La calibration par régression linéaire constitue la méthode standard en métrologie pour corriger ces biais.

## 1.2 Objectifs

- Charger et nettoyer les données du dataset UCI Air Quality
- Analyser le biais systématique du capteur `PT08.S1`
- Entraîner un modèle de régression linéaire sur 80 % des données
- Évaluer la performance sur 20 % de données inconnues
- Quantifier l'amélioration apportée par la correction

# 2 Dataset — Air Quality UCI

## 2.1 Description

Le dataset Air Quality UCI contient des mesures horaires de qualité de l'air collectées à Milan (Italie) entre mars 2004 et février 2005.

Table 1 – Colonnes principales du dataset

| Colonne | Description | Unité |
|---|---|---|
| Date | Date de la mesure | jj/mm/aaaa |
| Time | Heure de la mesure | hh.mm.ss |
| CO(GT) | Concentration CO (référence) | $mg/m^3$ |
| PT08.S1(CO) | Résistance capteur oxyde métallique | Ohms |

## 2.2 Valeurs manquantes

Dans ce dataset, la valeur `-200` indique une donnée manquante. Elle est remplacée par `NaN` avant traitement :

```
df.replace(-200, np.nan, inplace=True)
```

Listing 1 – Remplacement des valeurs manquantes

## 2.3 Pourquoi CO(GT) comme référence ?

- **GT = Ground Truth** : valeur certifiée par analyseur professionnel
- Seule colonne de vérité terrain pour le CO dans ce dataset
- Mesure la même grandeur physique que `PT08.S1`

# 3 Prétraitement des données

## 3.1 Chargement

```python
import pandas as pd
import numpy as np

df = pd.read_csv('AirQualityUCI.csv', sep=';', decimal=',')
df = df.dropna(axis=1, how='all')
df.replace(-200, np.nan, inplace=True)

paire = df[['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)']].dropna()
paire.columns = ['Date', 'Time', 'valeur_vraie', 'mesure_capteur']
```

Listing 2 – Chargement du fichier CSV

## 3.2 Normalisation

Les deux colonnes ont des unités très différentes :

Table 2 – Différence d'échelle entre les colonnes

| Colonne | Valeurs brutes | Après normalisation |
|---|---|---|
| CO(GT) | 1.6 – 11.9 mg/m$^3$ | 0.0 – 1.0 |
| PT08.S1 | 647 – 2040 Ohms | 0.0 – 1.0 |

La normalisation Min-Max est appliquée :

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

```python
from sklearn.preprocessing import MinMaxScaler

scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

mb_norm = scaler_X.fit_transform(
    mesures_biaisees.reshape(-1, 1)).flatten()
vv_norm = scaler_y.fit_transform(
    valeurs_vraies.reshape(-1, 1)).flatten()
```

Listing 3 – Normalisation Min-Max

# 4 Modèle de Machine Learning

## 4.1 Type de projet

Ce projet est un problème d'apprentissage supervisé de type régression :

- **Entrée X** : mesure normalisée du capteur `PT08.S1`
- **Sortie y** : valeur vraie normalisée `CO(GT)`
- **Algorithme** : Régression Linéaire

### 4.2 Split 80 % / 20 %

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Listing 4 – Séparation train/test

Table 3 – Répartition des données

| Ensemble | Proportion | Rôle |
|---|---|---|
| Train | 80 % | Entraînement du modèle |
| Test | 20 % | Évaluation sur données inconnues |

### 4.3 Régression Linéaire

Le modèle cherche la droite optimale :

$$\hat{y} = a \cdot x + b \tag{2}$$

où :

- $a$ = pente (coefficient directeur)
- $b$ = ordonnée à l'origine (biais)
- $x$ = mesure normalisée du capteur
- $\hat{y}$ = valeur corrigée estimée

```python
from sklearn.linear_model import LinearRegression

modele = LinearRegression()
modele.fit(X_train, y_train)

a = modele.coef_[0]
b = modele.intercept_
mesures_corrigees = modele.predict(X_test)
```

Listing 5 – Entraînement du modèle

## 5 Évaluation du modèle

### 5.1 Métriques utilisées

Table 4 – Métriques d'évaluation

| Métrique | Formule | Idéal |
|---|---|---|
| MAE | $\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$ | Proche de 0 |
| RMSE | $\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$ | Proche de 0 |
| $R^2$ | $1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$ | Proche de 1 |

### 5.2 Indicateurs de fiabilité

Table 5 – Grille d'interprétation du $R^2$

| Valeur $R^2$ | Interprétation |
|---|---|
| $R^2 = 1.00$ | Modèle parfait |
| $R^2 > 0.90$ | Modèle excellent |
| $R^2 > 0.70$ | Modèle acceptable |
| $R^2 < 0.50$ | Modèle faible |

## 6 Visualisations

Les 4 graphiques générés permettent d'analyser visuellement la calibration :

1. **Capteur vs Référence** : montre le décalage avant correction
2. **Distribution des erreurs** : visualise le biais systématique
3. **Modèle de correction** : affiche la droite de régression apprise
4. **Résidus avant/après** : compare l'amélioration de la correction

### 6.1 Code complet des graphiques

```python
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle(
    "Projet 5 - Calibration d'un capteur (Air Quality UCI)\n"
    "Capteur CO : PT08.S1 vs CO(GT) | PR. K. TLEMCANI - FSAC Hassan II",
    fontsize=12, fontweight='bold'
)

# Graphique 1 : Capteur vs Reference avant correction
ax1 = axes[0, 0]
ax1.scatter(y_test, X_test.flatten(), s=6, alpha=0.3,
            color='steelblue', label='Donnees test (20%)')
ax1.plot([0, 1], [0, 1], 'r--', linewidth=1.8,
         label='Reference ideale')
ax1.set_xlabel('Reference CO(GT) - normalisee')
```

```python
ax1.set_ylabel('Capteur PT08.S1 - normalise')
ax1.set_title('1 - Capteur vs Reference (avant correction)')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.4)

# Graphique 2 : Distribution des erreurs
ax2 = axes[0, 1]
erreurs = X_test.flatten() - y_test
erreur_moyenne = erreurs.mean()
ax2.hist(erreurs, bins=40, color='tomato',
         edgecolor='white', alpha=0.8)
ax2.axvline(erreur_moyenne, color='darkred', linestyle='--',
            linewidth=2,
            label=f'Erreur moy. = {erreur_moyenne:.4f}')
ax2.axvline(0, color='black', linestyle='-', linewidth=1)
ax2.set_xlabel('Erreur (capteur - reference)')
ax2.set_ylabel('Frequence')
ax2.set_title('2 - Distribution de l erreur systematique')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.4)

# Graphique 3 : Modele de correction
ax3 = axes[1, 0]
x_line = np.linspace(X_test.min(), X_test.max(), 300)
y_line = a * x_line + b
ax3.scatter(X_test.flatten(), y_test, s=6, alpha=0.3,
            color='steelblue', label='Donnees test')
ax3.plot(x_line, y_line, 'orange', linewidth=2.2,
         label=f'Modele : y = {a:.3f}x + {b:.3f}')
ax3.set_xlabel('Capteur PT08.S1 (normalise)')
ax3.set_ylabel('Reference CO(GT) (normalisee)')
ax3.set_title('3 - Ajustement du modele de correction')
ax3.legend()
ax3.grid(True, linestyle='--', alpha=0.4)

# Graphique 4 : Residus avant / apres correction
ax4 = axes[1, 1]
idx = np.arange(len(y_test))
ax4.plot(idx, X_test.flatten() - y_test,
         alpha=0.3, color='tomato',
         linewidth=0.7, label='Avant correction')
ax4.plot(idx, mesures_corrigees - y_test,
         alpha=0.5, color='seagreen',
         linewidth=0.7, label='Apres correction')
ax4.axhline(0, color='black', linewidth=1.2, linestyle='--')
ax4.set_xlabel('Index mesure')
ax4.set_ylabel('Residu (mesure - reference)')
ax4.set_title('4 - Residus : avant et apres correction')
ax4.legend()
ax4.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('calibration_capteur_UCI.png',
            dpi=150, bbox_inches='tight')
plt.show()
print('Figure sauvegardee : calibration_capteur_UCI.png')
```

Listing 6 – Génération des 4 graphiques de calibration

### 6.2 Analyse des graphiques

TABLE 6 – Interprétation des 4 graphiques

| N° | Ce qu'il montre |
|---|---|
| **1** | Les points sont au-dessus de la diagonale idéale ⇒ le capteur **surestime** la concentration de CO |
| **2** | L'erreur moyenne de 0.1635 confirme un biais systématique positif constant |
| **3** | La droite $y = 0.687x - 0.056$ ajuste bien les données ⇒ relation linéaire confirmée |
| **4** | Les résidus verts (après correction) sont nettement plus proches de 0 que les rouges |

## 7 Bibliothèques Python utilisées

Cette section présente en détail chaque bibliothèque Python utilisée dans le projet, son rôle, ses fonctions clés et des exemples d'utilisation concrets.

### 7.1 NumPy — Calcul numérique

**NumPy `import numpy as np`**

**Qu'est-ce que NumPy ?**
NumPy (*Numerical Python*) est la bibliothèque fondamentale du calcul scientifique en Python. Elle fournit un objet tableau multidimensionnel (`ndarray`) et un ensemble de fonctions mathématiques performantes opérant sur ces tableaux.

**Installation :**

```
pip install numpy
```

**Fonctions utilisées dans ce projet :**

| Fonction | Rôle |
|---|---|
| `np.nan` | Constante représentant une valeur manquante (Not a Number) |
| `np.arange(n)` | Crée un tableau d'indices entiers de 0 à n-1 |
| `np.linspace(a,b,n)` | Crée n valeurs régulièrement espacées entre a et b |
| `arr.reshape(-1,1)` | Redimensionne un tableau 1D en colonne 2D |
| `arr.flatten()` | Aplatit un tableau multidimensionnel en 1D |

**Exemple concret (ce projet) :**

```python
import numpy as np

# Remplacer -200 par NaN (valeur manquante du dataset)
```

```python
df.replace(-200, np.nan, inplace=True)

# Creer les indices pour l'axe x du graphique 4
idx = np.arange(len(y_test))

# Creer 300 points equidistants pour tracer la droite de regression
x_line = np.linspace(X_test.min(), X_test.max(), 300)
```

Listing 7 – Utilisation de NumPy dans le projet

## 7.2 Pandas — Manipulation de données

**Pandas `import pandas as pd`**

**Qu'est-ce que Pandas ?**
Pandas est la bibliothèque standard de manipulation et d'analyse de données tabulaires en Python. Elle introduit deux structures principales : `Series` (tableau 1D étiqueté) et `DataFrame` (tableau 2D similaire à un tableur Excel ou une table SQL).

**Installation :**

```
pip install pandas
```

**Fonctions utilisées dans ce projet :**

| Fonction | Rôle |
|---|---|
| `pd.read_csv()` | Lit un fichier CSV et retourne un DataFrame |
| `df.replace()` | Remplace des valeurs dans le DataFrame |
| `df.dropna()` | Supprime les lignes/colonnes contenant des NaN |
| `df[colonnes]` | Sélection de colonnes spécifiques |
| `df.columns = []` | Renomme les colonnes du DataFrame |

**Paramètres clés de `read_csv` :**

| Paramètre | Description |
|---|---|
| `sep=';'` | Le séparateur de colonnes est le point-virgule |
| `decimal=','` | La virgule est utilisée comme séparateur décimal |

**Exemple concret (ce projet) :**

```python
import pandas as pd

# Charger le fichier CSV avec separateur ; et decimal ,
df = pd.read_csv('AirQualityUCI.csv', sep=';', decimal=',')

# Supprimer les colonnes entierement vides
df = df.dropna(axis=1, how='all')
```

```python

# Remplacer -200 (code de donnee manquante) par NaN
df.replace(-200, np.nan, inplace=True)

# Selectionner les 4 colonnes utiles et supprimer les lignes incompletes
paire = df[['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)']].dropna()

# Renommer les colonnes pour plus de clarte
paire.columns = ['Date', 'Time', 'valeur_vraie', 'mesure_capteur']
```

Listing 8 – Utilisation de Pandas dans le projet

### 7.3 Scikit-learn — Machine Learning

**Scikit-learn `import sklearn`**

**Qu'est-ce que Scikit-learn ?**
Scikit-learn est la bibliothèque de Machine Learning la plus utilisée en Python. Elle offre des outils simples et efficaces pour la classification, la régression, le clustering, la réduction de dimensionnalité et le prétraitement des données. Elle est construite sur NumPy, SciPy et Matplotlib.

**Installation :**

```
pip install scikit-learn
```

**Trois modules utilisés dans ce projet :**

**Module 1 : `MinMaxScaler` — Normalisation**

| Méthode | Rôle |
|---|---|
| `MinMaxScaler()` | Crée un objet de normalisation Min-Max |
| `.fit_transform(X)` | Calcule min/max **et** transforme les données |
| `.inverse_transform(X)` | Remet les données dans leur échelle d'origine |

```python
from sklearn.preprocessing import MinMaxScaler

scaler_X = MinMaxScaler()    # Scaler pour la variable X (capteur)
scaler_y = MinMaxScaler()    # Scaler pour la variable y (reference)

# fit_transform : apprend le min/max ET normalise en une seule etape
mb_norm = scaler_X.fit_transform(
    mesures_biaisees.reshape(-1, 1)).flatten()
vv_norm = scaler_y.fit_transform(
    valeurs_vraies.reshape(-1, 1)).flatten()
```

Listing 9 – Normalisation avec MinMaxScaler

**Module 2 : `train_test_split` — Séparation train/test**

| Paramètre | Rôle |
|---|---|
| `test_size=0.2` | 20 % des données pour le test |
| `random_state=42` | Graine aléatoire pour la reproductibilité |

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# X_train : 80% des donnees d'entree  -> entraine le modele
# X_test  : 20% des donnees d'entree  -> evalue le modele
# y_train : 80% des donnees de sortie -> valeurs cibles d'entrainement
# y_test  : 20% des donnees de sortie -> valeurs cibles de test
```

Listing 10 – Séparation des données

**Module 3 : `LinearRegression` — Modèle de régression**

| Méthode / Attribut | Rôle |
|---|---|
| `LinearRegression()` | Crée le modèle de régression linéaire |
| `.fit(X, y)` | Entraîne le modèle sur les données |
| `.predict(X)` | Prédit les valeurs pour de nouvelles données |
| `.coef_[0]` | Retourne la pente $a$ de la droite |
| `.intercept_` | Retourne le biais $b$ de la droite |

```python
from sklearn.linear_model import LinearRegression

modele = LinearRegression()

# .fit() minimise la somme des carres des residus (methode des MCO)
modele.fit(X_train, y_train)

a = modele.coef_[0]      # Pente de la droite de regression
b = modele.intercept_    # Ordonnee a l'origine

# .predict() applique la formule : y_pred = a * X + b
mesures_corrigees = modele.predict(X_test)
```

Listing 11 – Entraînement et prédiction

## 7.4 Matplotlib — Visualisation

**Matplotlib `import matplotlib.pyplot as plt`**

**Qu'est-ce que Matplotlib ?**
Matplotlib est la bibliothèque de référence pour la création de graphiques et de visualisations en Python. Elle permet de produire des figures de qualité publication : courbes, histogrammes, nuages de points, diagrammes en barres, etc. Le module `pyplot` fournit une interface similaire à MATLAB.

**Installation :**

```
pip install matplotlib
```

**Fonctions utilisées dans ce projet :**

| Fonction | Rôle |
|---|---|
| `plt.subplots(2,2)` | Crée une grille de 4 sous-graphiques (2 lignes × 2 colonnes) |
| `fig.suptitle()` | Titre global de la figure |
| `ax.scatter()` | Nuage de points (données brutes) |
| `ax.plot()` | Ligne / courbe (droite de régression) |
| `ax.hist()` | Histogramme (distribution des erreurs) |
| `ax.axvline()` | Ligne verticale (erreur moyenne) |
| `ax.axhline()` | Ligne horizontale (résidu = 0) |
| `ax.set_xlabel()` | Étiquette de l'axe x |
| `ax.set_ylabel()` | Étiquette de l'axe y |
| `ax.set_title()` | Titre du sous-graphique |
| `ax.legend()` | Affiche la légende |
| `ax.grid()` | Affiche la grille de fond |
| `plt.tight_layout()` | Ajuste automatiquement les marges |
| `plt.savefig()` | Sauvegarde la figure en fichier image |

**Exemple concret (ce projet) :**

```python
import matplotlib.pyplot as plt

# Creer une figure avec 4 sous-graphiques (2 lignes, 2 colonnes)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# axes est une matrice 2x2 :
# axes[0,0] = graphique haut-gauche  (Capteur vs Reference)
# axes[0,1] = graphique haut-droite  (Distribution erreurs)
# axes[1,0] = graphique bas-gauche   (Modele de correction)
# axes[1,1] = graphique bas-droite   (Residus)

# Nuage de points : donnees de test
axes[0,0].scatter(y_test, X_test.flatten(),
                  s=6,        # taille des points
                  alpha=0.3,  # transparence 30%
                  color='steelblue')

# Sauvegarder en PNG haute resolution (150 dpi)
plt.savefig('calibration_capteur_UCI.png',
            dpi=150, bbox_inches='tight')
```

Listing 12 – Structure des sous-graphiques

### 7.5 Récapitulatif des bibliothèques

Table 7 – Vue d'ensemble des bibliothèques du projet

| Bibliothèque | Version typique | Rôle principal | Import |
|---|---|---|---|
| NumPy | 1.24+ | Calcul numérique, tableaux | `import numpy as np` |
| Pandas | 2.0+ | Manipulation de données | `import pandas as pd` |
| Scikit-learn | 1.3+ | Machine Learning | `import sklearn` |
| Matplotlib | 3.7+ | Visualisation graphique | `import matplotlib.pyplot` |

**Installation de toutes les bibliothèques en une seule commande :**

```
pip install numpy pandas scikit-learn matplotlib
```

## 8 Conclusion

Ce projet démontre qu'une régression linéaire simple suffit à corriger efficacement le biais systématique d'un capteur bas coût :

- La relation entre `PT08.S1` et `CO(GT)` est linéaire
- Le modèle est entraîné sur 80 % des données
- Évalué sur 20 % de données inconnues (validation réelle)
- Le $R^2$ proche de 1 confirme la qualité du modèle
- L'amélioration MAE/RMSE quantifie le gain de précision

La formule de calibration finale est :

$$\texttt{valeur_corrigee} = a \times \texttt{x} + b \tag{3}$$

Cette approche constitue la méthode standard en métrologie pour l'étalonnage de capteurs environnementaux bas coût.

## 8 Références

[1] UCI Machine Learning Repository — Air Quality Dataset.
https://archive.ics.uci.edu/ml/datasets/Air+Quality

[2] Scikit-learn Documentation — LinearRegression.
https://scikit-learn.org/stable/modules/linear_model.html

[3] De Vito, S. et al. (2008). On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario. *Sensors and Actuators B : Chemical*, 129(2), 750–757.

[4] NumPy Documentation.
https://numpy.org/doc/stable/

[5] Pandas Documentation.
https://pandas.pydata.org/docs/

[6] Matplotlib Documentation.
https://matplotlib.org/stable/
https://github.com/khalifahte34-dev/Projet-PHEI---Groupe-7